Université TEK-UP

Module Cybersécurité – Projet 2

RAPPORT DE CONCEPTION
et de développement

SecureAuth

Coffre-fort de documents classifiés :
système sécurisé d'authentification et de cryptographie

Équipe de développement

Khalil Hnich
Houssem Ben Khamou

Encadrant

Oussema Bakali

Année universitaire 2026–2027

Octobre 2026

Table des matières

1 Introduction et Contexte du Projet **1**
- 1.1 Introduction . . . 1
- 1.2 Problématique . . . 1
- 1.3 Besoins fonctionnels . . . 1
- 1.4 Besoins non fonctionnels . . . 2

2 Organisation du Projet **3**
- 2.1 Démarche . . . 3
- 2.2 Organisation de l'équipe . . . 4

3 Architecture et Conception du Système **5**
- 3.1 Architecture globale . . . 5
- 3.2 Diagramme de Classes . . . 5
- 3.3 Diagramme de Cas d'Utilisation . . . 7
- 3.4 Diagramme de Séquence : Inscription . . . 8
- 3.5 Technologies Utilisées . . . 9

4 Captures d'Écran de l'Application **10**
- 4.1 Page d'Inscription . . . 10
- 4.2 Page de Connexion . . . 11
- 4.3 Tableau de Bord . . . 13
- 4.4 Changement de Mot de Passe . . . 14
- 4.5 Espace Documents . . . 15
- 4.6 Espace Administration . . . 20

5 Tests de Sécurité **26**
- 5.1 Méthode . . . 26
- 5.2 Résultats . . . 26
- 5.3 Défauts découverts et corrigés . . . 28
- 5.4 Durcissement . . . 29

6 Conclusion et Perspectives **30**
- 6.1 Bilan du Projet . . . 30
- 6.2 Leçons Apprises . . . 31
- 6.3 Perspectives d'Évolution . . . 31
- 6.4 Conclusion Générale . . . 32

A Glossaire **33**

Table des figures

3.1 Architecture de déploiement : quatre conteneurs et deux réseaux 5
3.2 Diagramme de classes – SecureAuth 6
3.3 Diagramme de cas d'utilisation – SecureAuth 7
3.4 Diagramme de séquence – Inscription 8
3.5 Technologies utilisées dans le projet 9

4.1 Formulaire d'inscription 10
4.2 Page de connexion 11
4.3 Échec de connexion : message unique 12
4.4 Tableau de bord d'un agent connecté 13
4.5 Changement de mot de passe 14
4.6 Liste des documents lisibles par alice 15
4.7 Lecture d'un document TRÈS SECRET 16
4.8 Création d'un document 17
4.9 Dépôt d'un fichier PDF, DOC ou DOCX 17
4.10 Fiche d'un document avec fichier joint 18
4.11 Liste d'accès d'un document (partage) 19
4.12 Accès refusé : page 404 identique à un document inexistant 19
4.13 Comptes et habilitations 20
4.14 Ajout d'un utilisateur par l'administrateur 21
4.15 Dépôt par l'administrateur : classification TRÈS SECRET, puis partage . 22
4.16 Documents : métadonnées uniquement 23
4.17 Journal d'audit 23
4.18 Vérification du journal : chaîne intègre 24
4.19 Vérification du journal après modification d'une ligne en base 25

Liste des tableaux

2.1 Planning du projet et état d'avancement 4

5.1 Les trente cas de test de sécurité (T26 à T30 : ajouts de l'équipe) 26

Chapitre 1

Introduction et Contexte du Projet

1.1 Introduction

SecureAuth est une application Web qui protège les documents classifiés d'une agence fictive. Elle décide côté serveur qui a le droit de lire quel document, elle chiffre chaque document avant de le ranger en base, et elle note chaque passage devant un contrôle d'accès dans un journal que l'on ne peut pas modifier sans que cela se voie.

Le projet est le sujet n°2 du module Cybersécurité de TEK-UP, intitulé « Système sécurisé d'authentification et de cryptographie ». Il se déroule sur six semaines. L'idée de départ est simple : se connecter avec un mot de passe ne suffit pas à protéger des informations sensibles. Le vrai travail est de décider ce que chaque utilisateur peut lire une fois connecté, de le faire respecter par le serveur, et de pouvoir le prouver.

Chaque document porte un niveau de classification (PUBLIC, CONFIDENTIEL, SECRET ou TRÈS SECRET) et chaque agent porte une habilitation sur la même échelle. Lire un document demande deux choses : une habilitation au moins égale à sa classification, et la présence de l'agent sur la liste d'accès du document. L'administrateur gère les comptes mais ne lit aucun document.

1.2 Problématique

Un système qui manipule des documents sensibles est exposé à plusieurs menaces. Un mot de passe peut être deviné ou volé, un cookie de session peut être rejoué, un utilisateur peut essayer de lire un document qui ne lui est pas destiné en modifiant simplement l'adresse dans son navigateur. Un administrateur trop puissant est lui aussi un risque : s'il peut tout lire, la compromission de son compte compromet tout. Enfin, quelqu'un qui accède directement à la base de données peut lire les documents ou effacer les traces de ses actions.

La question à laquelle le projet répond est donc la suivante : comment construire une application où l'authentification, l'autorisation et la cryptographie se complètent, de façon qu'aucune de ces protections ne repose sur une seule barrière ?

1.3 Besoins fonctionnels

Le système doit permettre :

— à un visiteur de s'inscrire avec un nom d'utilisateur, une adresse e-mail et un mot de passe, puis de se connecter, de se déconnecter et de changer son mot de passe ;

— de créer le premier administrateur uniquement en ligne de commande, jamais depuis l'interface ;
— de verrouiller un compte pendant quinze minutes après cinq échecs de connexion consécutifs ;
— à un agent de créer, de lister et de lire des documents texte classifiés, dans la limite de son habilitation ;
— à un agent de partager un document qu'il a créé en ajoutant ou en retirant des agents de sa liste d'accès ;
— à l'administrateur de lister les comptes, de modifier une habilitation, d'activer ou de désactiver un compte, et de consulter la liste d'accès d'un document sans jamais voir son contenu ;
— à l'administrateur de consulter le journal d'audit et d'en vérifier l'intégrité ;
— d'enregistrer dans le journal les événements de sécurité (inscription, connexion, échec, lecture, refus d'accès, changement d'habilitation, etc.) sans y écrire de mot de passe, de contenu de document ni de clé.

1.4 Besoins non fonctionnels

— Une vérification de mot de passe prend à peu près le même temps que le compte existe ou non, pour ne pas révéler les comptes valides.
— Un document non autorisé donne une réponse 404, jamais 403 : le système ne confirme pas l'existence d'un document classifié.
— Le journal reste cohérent même avec plusieurs processus qui écrivent en parallèle.
— L'application ne tourne pas en *root* et la base n'est jamais accessible depuis l'extérieur.
— Les secrets (clés AES et HMAC, mots de passe de la base) ne sont jamais dans le dépôt Git ; l'application refuse de démarrer si une clé manque ou est mal formée.
— Le déploiement se fait avec Docker Compose, sous Windows comme sous Linux.
— Les tests automatiques s'exécutent sans conteneur, avec une base en mémoire et un faux Redis.
— Le code est modulaire : toutes les décisions d'accès aux documents passent par un seul module.

Chapitre 2

Organisation du Projet

2.1 Démarche

Le sujet impose six phases, une par semaine. L'équipe les a suivies dans l'ordre. Une fonctionnalité de sécurité est considérée comme terminée lorsqu'elle a un test automatique qui passe, une vérification manuelle décrite avec son résultat attendu, et une preuve utilisable dans le rapport (capture, extrait du journal ou sortie de commande). Cette règle vient du module : un résultat de sécurité doit être démontré, pas seulement affirmé. Le dépôt contient aujourd'hui plus de trois cents tests automatiques et un script qui rejoue les attaques contre l'application réellement déployée.

Les décisions ont été prises avant de coder et consignées dans un cahier des charges (version 1.0, octobre 2026) : exigences fonctionnelles, exigences de sécurité, modèle de menace avec quatorze menaces et plan de vingt-cinq tests.

Semaine	Phase	État actuel
1	Conception et infrastructure	Terminée : Docker Compose, Nginx avec TLS, mini-PKI, MariaDB, Redis.
2	Authentification	Terminée : inscription, connexion, déconnexion, mot de passe, sessions Redis, verrouillage après cinq échecs, journal d'audit.
3	Autorisation et chiffrement	Terminée : règles d'accès, AES-256-GCM, espaces documents et administration, jeu de démonstration, ajout d'utilisateurs par l'administrateur, dépôt de fichiers PDF, DOC et DOCX.
4	Tests de sécurité	Réalisés : les vingt-cinq cas, plus cinq cas pour les comptes créés par l'administrateur et les fichiers, par tests automatiques et par un script contre les vrais services.
5	Corrections et durcissement	Terminée : limitation de débit, HSTS, conteneur applicatif en lecture seule, certificat corrigé.
6	Livraison	Terminée : README, tests de sécurité, scénario de démonstration, rapport.

TABLE 2.1 – Planning du projet et état d'avancement

2.2 Organisation de l'équipe

L'équipe est composée de deux étudiants, Khalil Hnich et Houssem Ben Khamou, encadrés par Oussema Bakali. Tout le code est versionné dans un dépôt Git. Les deux membres participent à la conception, au développement et aux tests ; les choix techniques importants sont écrits dans le cahier des charges plutôt que laissés à la mémoire de chacun.

Chapitre 3

Architecture et Conception du Système

3.1 Architecture globale

L'application tourne dans quatre conteneurs Docker. Le navigateur ne parle qu'à Nginx, qui est le seul service dont les ports (80 et 443) sont publiés sur la machine. Nginx termine le chiffrement TLS puis transmet la requête à l'application Flask, servie par Gunicorn. Seule l'application peut joindre MariaDB et Redis, qui se trouvent sur un second réseau Docker déclaré « interne » : ils n'ont aucun accès à Internet.

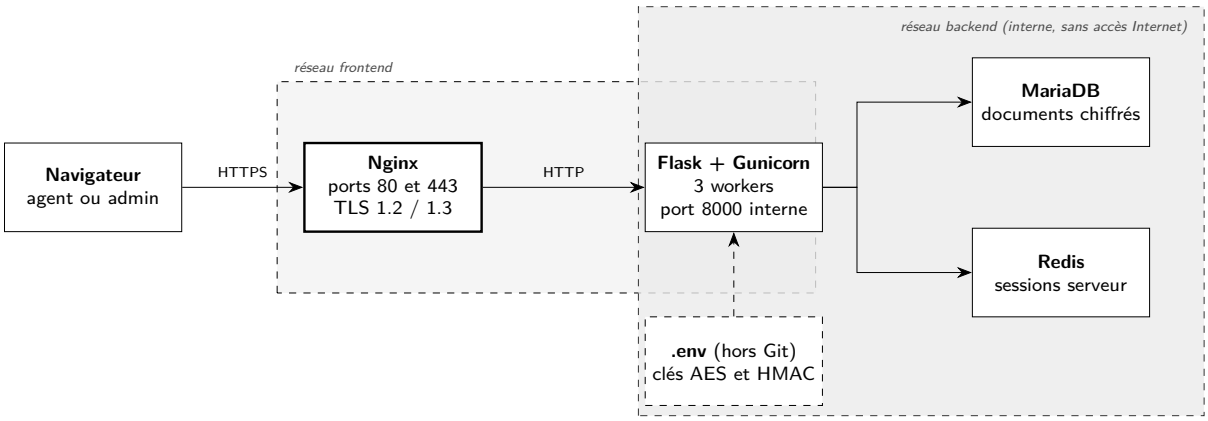


FIGURE 3.1 – Architecture de déploiement : quatre conteneurs et deux réseaux

Les secrets sont injectés uniquement là où ils servent. L'application ne reçoit jamais le mot de passe *root* de la base, et les clés de chiffrement viennent d'un fichier `.env` exclu de Git.

3.2 Diagramme de Classes

Le diagramme de classes montre les cinq tables de la base et leurs relations. Seule la référence d'un document reste en clair : son titre et son contenu sont chiffrés ensemble.

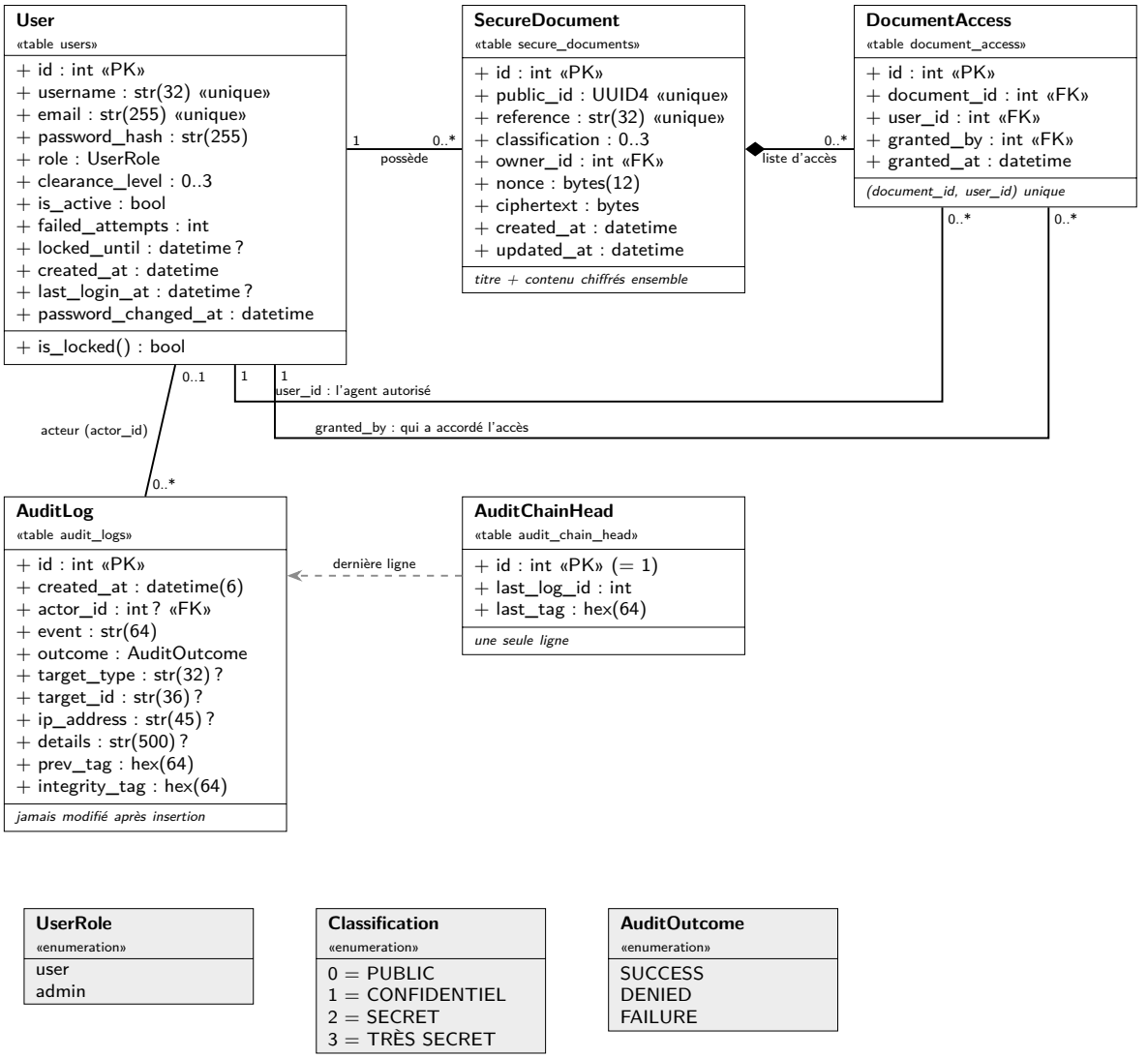


FIGURE 3.2 – Diagramme de classes – SecureAuth

Classes principales :

- **User** : un compte. Il garde l'empreinte Argon2id du mot de passe (jamais le mot de passe), son rôle (`user` ou `admin`), son habilitation de 0 à 3, son état actif ou désactivé, et les champs nécessaires au verrouillage (`failed_attempts`, `locked_until`).
- **SecureDocument** : un document classifié. Il a un identifiant public aléatoire (UUID) utilisé dans les adresses, une référence lisible, une classification, un propriétaire, et le texte chiffré avec son nonce de 12 octets.
- **DocumentAccess** : une ligne par agent autorisé à lire un document (le besoin d'en connaître). Elle garde aussi qui a accordé l'accès et quand.
- **AuditLog** : une ligne du journal. Chaque ligne contient l'empreinte de la précédente et sa propre empreinte HMAC-SHA256.
- **AuditChainHead** : une seule ligne qui mémorise la dernière entrée du journal. Elle permet de détecter la suppression des dernières lignes.

Un compte n'est jamais supprimé, seulement désactivé : supprimer l'auteur d'une ligne déjà signée casserait la signature du journal. De même, la date du journal est stockée avec les microsecondes, sinon la date relue en base serait différente de la date signée.

3.3 Diagramme de Cas d'Utilisation

Le diagramme de cas d'utilisation présente ce que chaque acteur peut faire. Les ellipses grises sont des cas internes, réutilisés par plusieurs fonctionnalités.

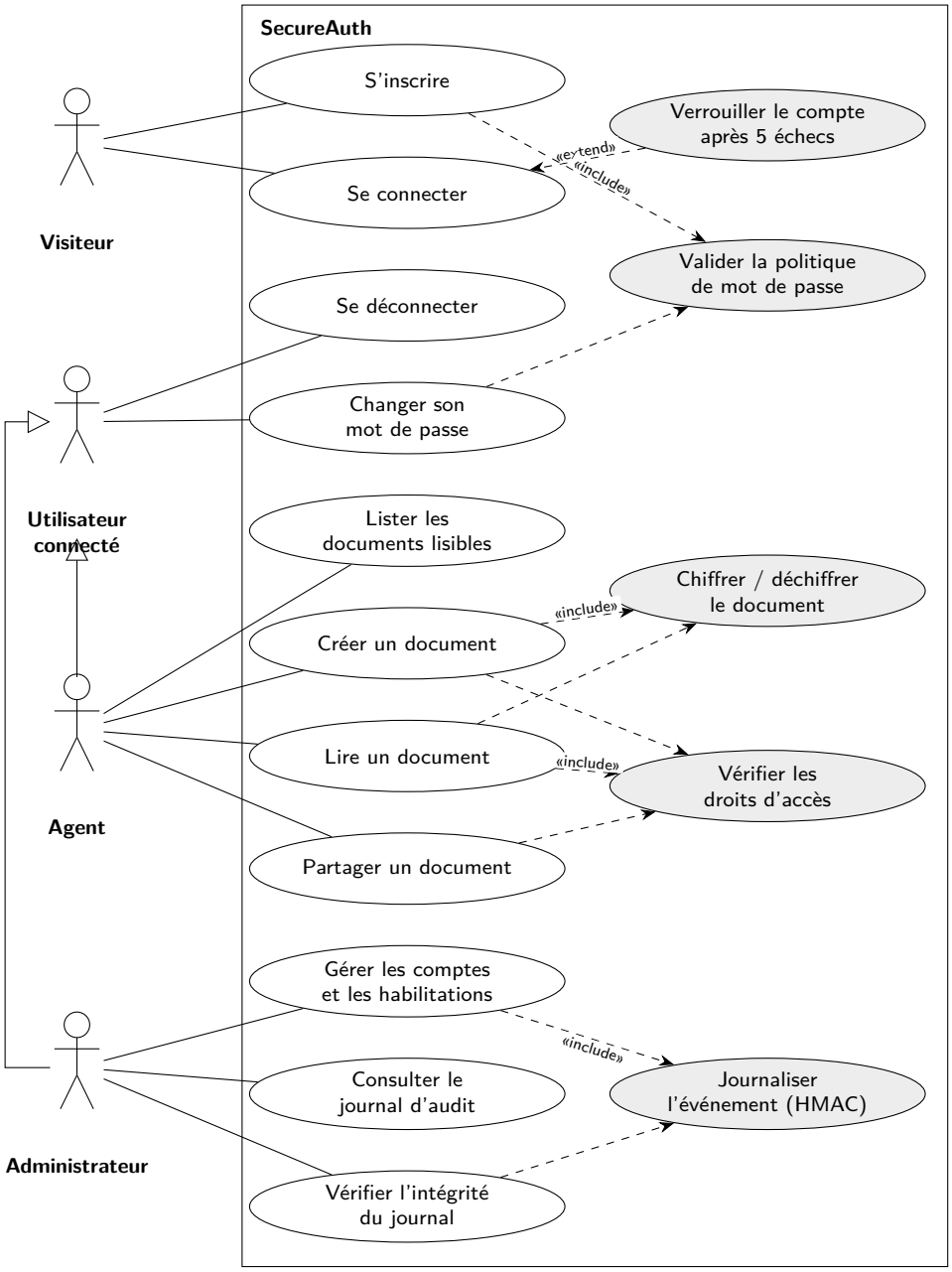


FIGURE 3.3 – Diagramme de cas d'utilisation – SecureAuth

Cas d'utilisation principaux :

— S'inscrire, se connecter, se déconnecter et changer son mot de passe ;
— lister, créer et lire des documents selon son habilitation ;
— partager un document que l'on a créé ;
— gérer les comptes et les habilitations (administrateur) ;
— consulter le journal d'audit et en vérifier l'intégrité (administrateur).

Trois points méritent d'être précisés. Lire un document passe toujours par le cas interne « Vérifier les droits d'accès » : c'est le point de décision unique du projet. L'administrateur

n'est associé à aucun cas de lecture de document, ce qui traduit la séparation des rôles. Enfin, le verrouillage du compte est une extension de la connexion : il ne se déclenche qu'après cinq échecs consécutifs.

3.4 Diagramme de Séquence : Inscription

Le diagramme de séquence suivant détaille ce qui se passe quand un visiteur envoie le formulaire d'inscription.

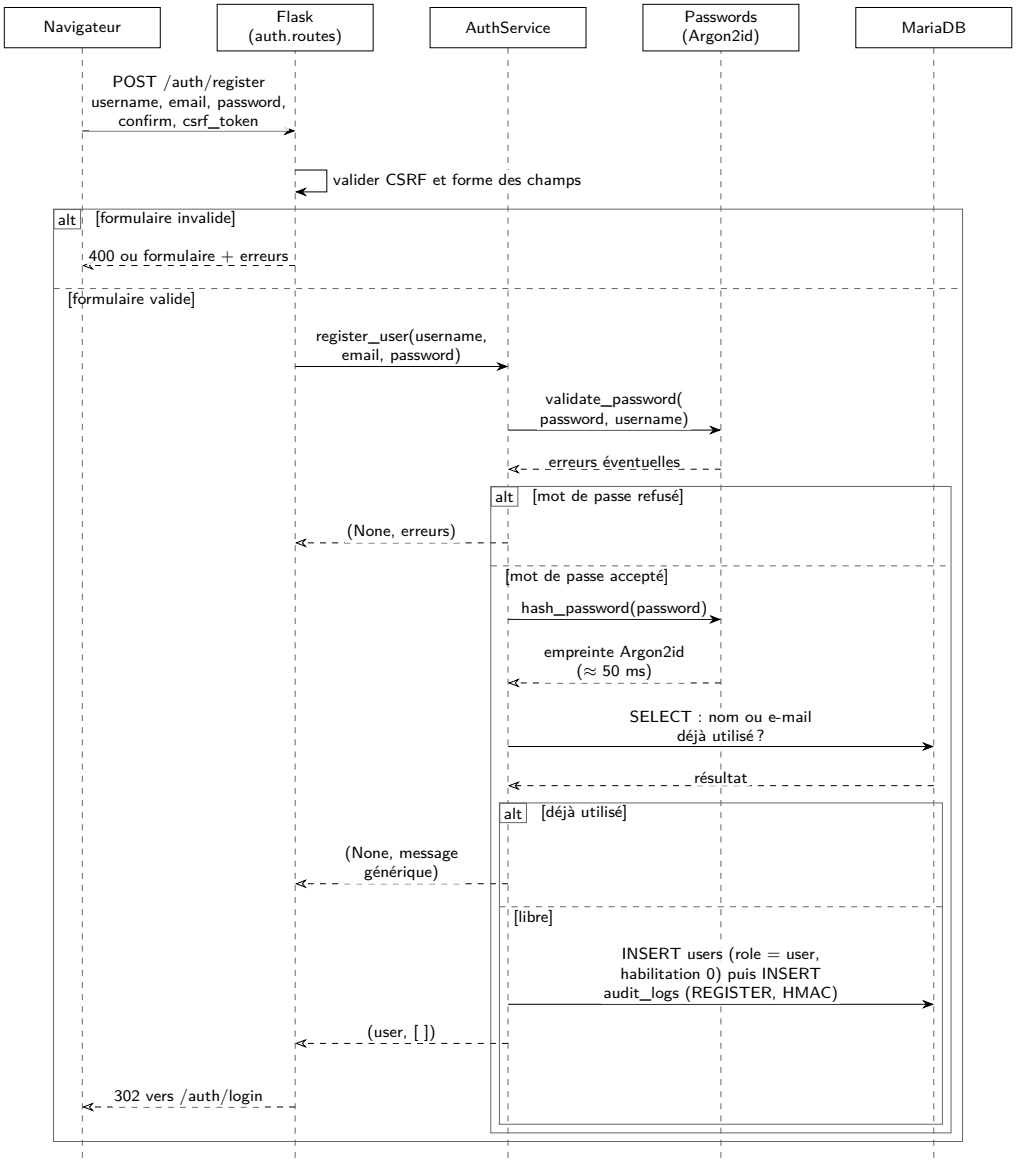


FIGURE 3.4 – Diagramme de séquence – Inscription

Description du fonctionnement :

Le navigateur envoie le formulaire avec son jeton CSRF. La route Flask vérifie d'abord le jeton et la forme des champs ; si quelque chose ne va pas, le formulaire est renvoyé avec les erreurs. Sinon, elle passe les données au service d'authentification, qui commence par vérifier la politique de mot de passe : entre 12 et 128 caractères, différent du nom d'utilisateur, absent d'une liste de mots de passe courants.

Le service calcule ensuite l'empreinte Argon2id, *avant* de regarder si le nom ou l'adresse e-mail existent déjà. C'est volontaire : l'opération dure environ 50 ms dans tous les cas, donc le temps de réponse ne révèle pas si un compte existe. Si le nom ou l'adresse sont déjà pris, l'utilisateur reçoit un message unique et volontairement vague (« Inscription impossible avec ces informations »).

Si tout est correct, le compte est créé avec le rôle `user` et l'habilitation 0, et une ligne `REGISTER` est ajoutée au journal d'audit, signée par HMAC. L'utilisateur est redirigé vers la page de connexion. Un nouvel inscrit ne peut donc jamais devenir administrateur par ce formulaire.

3.5 Technologies Utilisées

La pile technique a été fixée au début du projet et n'a pas changé.

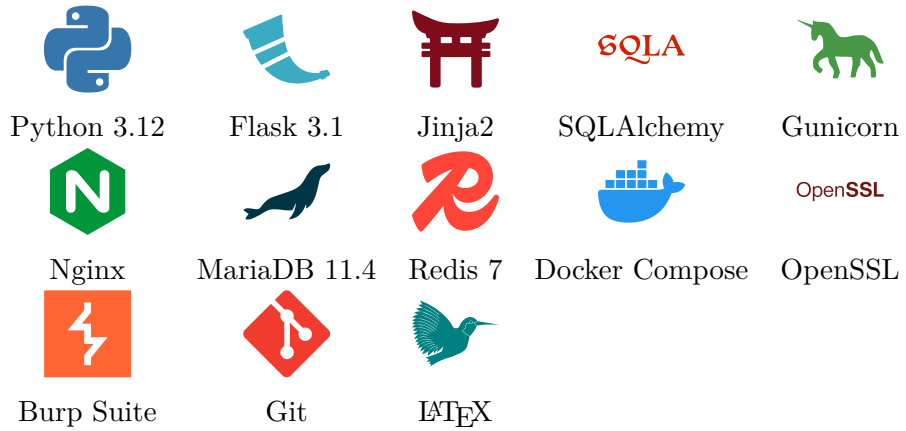


FIGURE 3.5 – Technologies utilisées dans le projet

— **Application** : Python et Flask, avec Jinja2 pour les pages et SQLAlchemy pour accéder à la base. Aucune requête SQL n'est construite à la main, ce qui écarte l'injection SQL. Les formulaires utilisent Flask-WTF (jeton CSRF), les sessions Flask-Session et la limitation de débit Flask-Limiter.
— **Serveurs** : Gunicorn (trois processus, utilisateur non-root) exécute Flask. Nginx est le point d'entrée : TLS 1.2 et 1.3, redirection de HTTP vers HTTPS, en-têtes de sécurité.
— **Données** : MariaDB stocke les comptes, les documents chiffrés et le journal. Redis stocke les sessions côté serveur : le cookie ne contient qu'un identifiant aléatoire, et la déconnexion détruit réellement la session.
— **Cryptographie** : `argon2-cffi` pour les mots de passe (Argon2id), `cryptography` pour AES-256-GCM, le module standard `hmac` pour le journal. Aucun algorithme maison. OpenSSL sert à créer la mini-autorité de certification et le certificat du serveur.
— **Déploiement** : Docker Compose, avec deux réseaux dont un interne.
— **Tests** : `pytest` pour les tests automatiques ; Burp Suite, OpenSSL, curl et les outils de développement du navigateur pour les tests de sécurité de la semaine 4.
— **Documentation** : Git pour le dépôt, LaTeX pour ce rapport et ses diagrammes.

Les icônes des logos proviennent du projet Simple Icons ; les marques appartiennent à leurs propriétaires respectifs.

Chapitre 4

Captures d'Écran de l'Application

Les captures de ce chapitre montrent les pages déjà réalisées. Elles ont été prises sur une instance locale avec un compte de démonstration (`alice`, habilitation SECRET) créé pour l'occasion.

4.1 Page d'Inscription

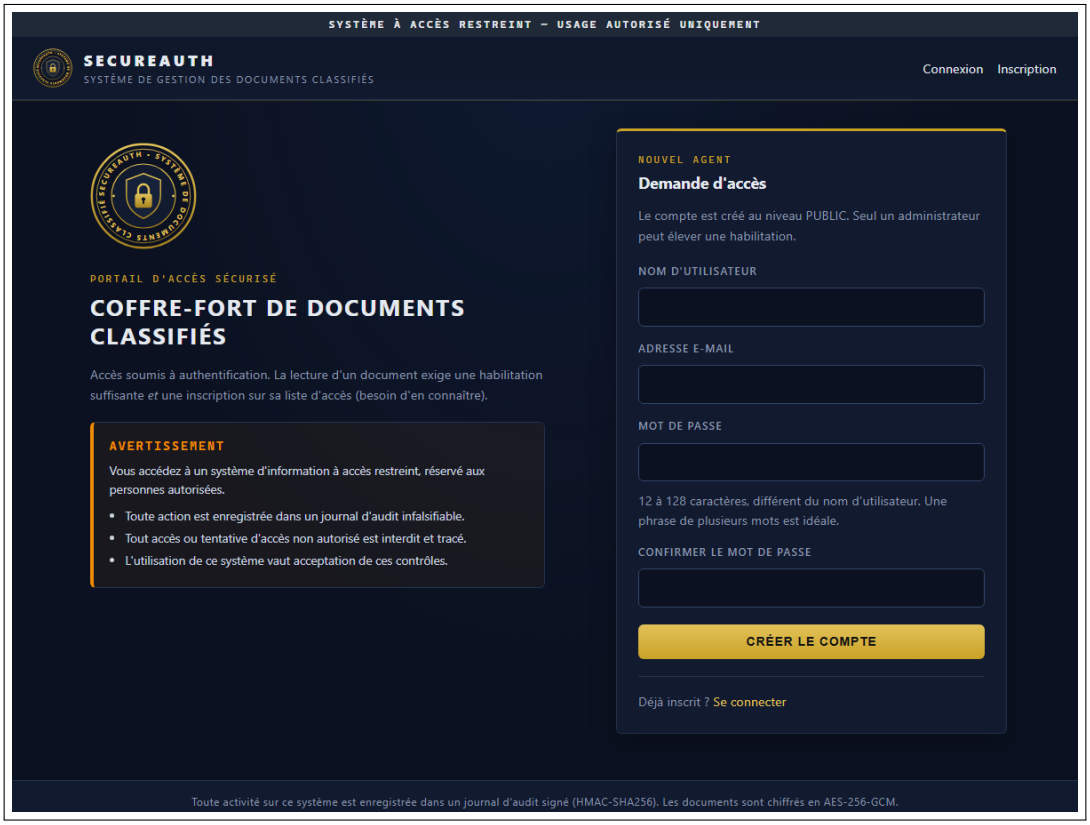


FIGURE 4.1 – Formulaire d'inscription

Le formulaire demande le nom d'utilisateur, l'adresse e-mail, le mot de passe et sa confirmation. La page rappelle la règle de longueur (12 à 128 caractères) et précise que le compte est créé au niveau PUBLIC : seul un administrateur peut élever une habilitation. Un avertissement rappelle que toute action est enregistrée dans le journal d'audit.

4.2 Page de Connexion

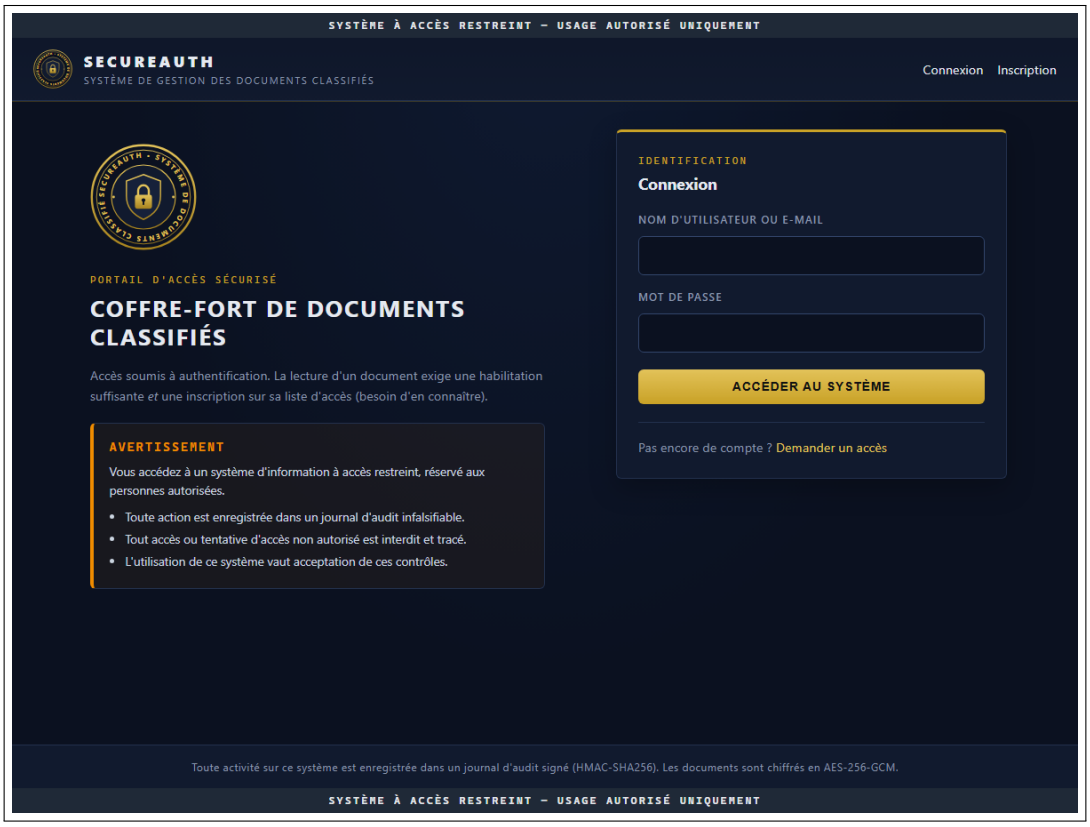


FIGURE 4.2 – Page de connexion

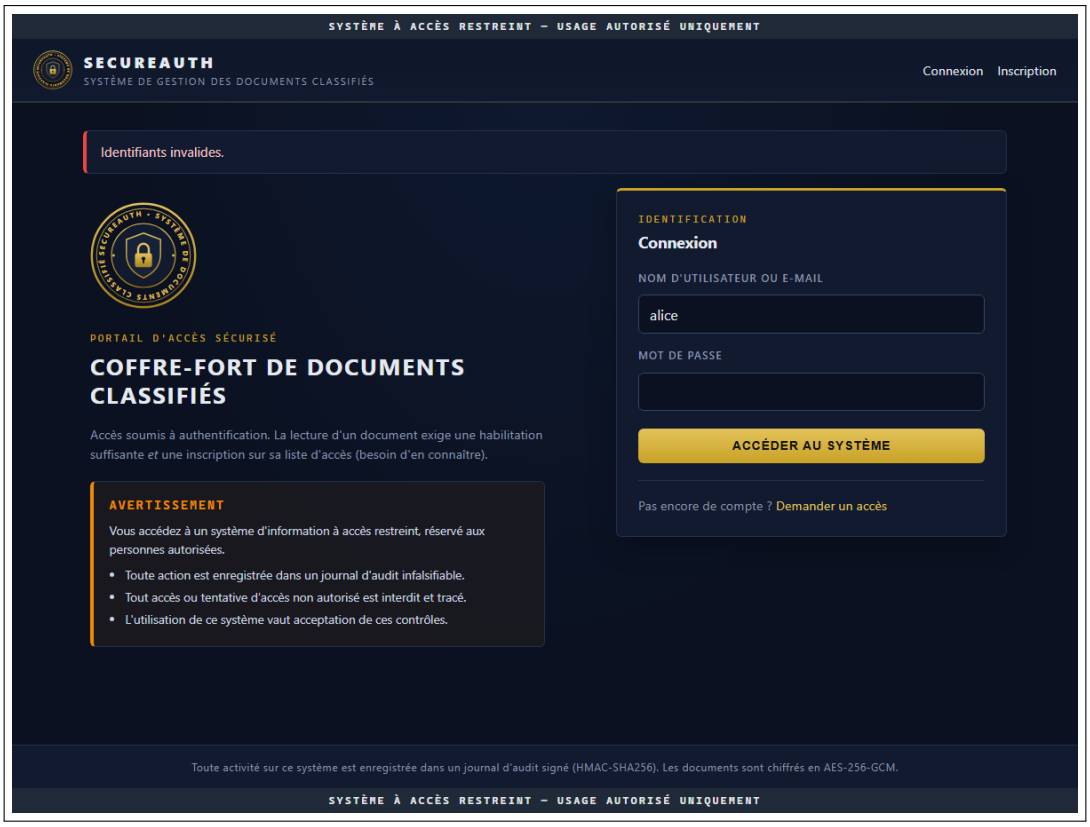


FIGURE 4.3 – Échec de connexion : message unique

On peut se connecter avec le nom d'utilisateur ou l'adresse e-mail. Quand l'identifiant est inconnu, le compte désactivé ou le mot de passe faux, la page affiche toujours le même message, « Identifiants invalides », afin qu'un attaquant ne puisse pas deviner quels comptes existent. Le motif réel est conservé uniquement dans le journal d'audit.

4.3 Tableau de Bord

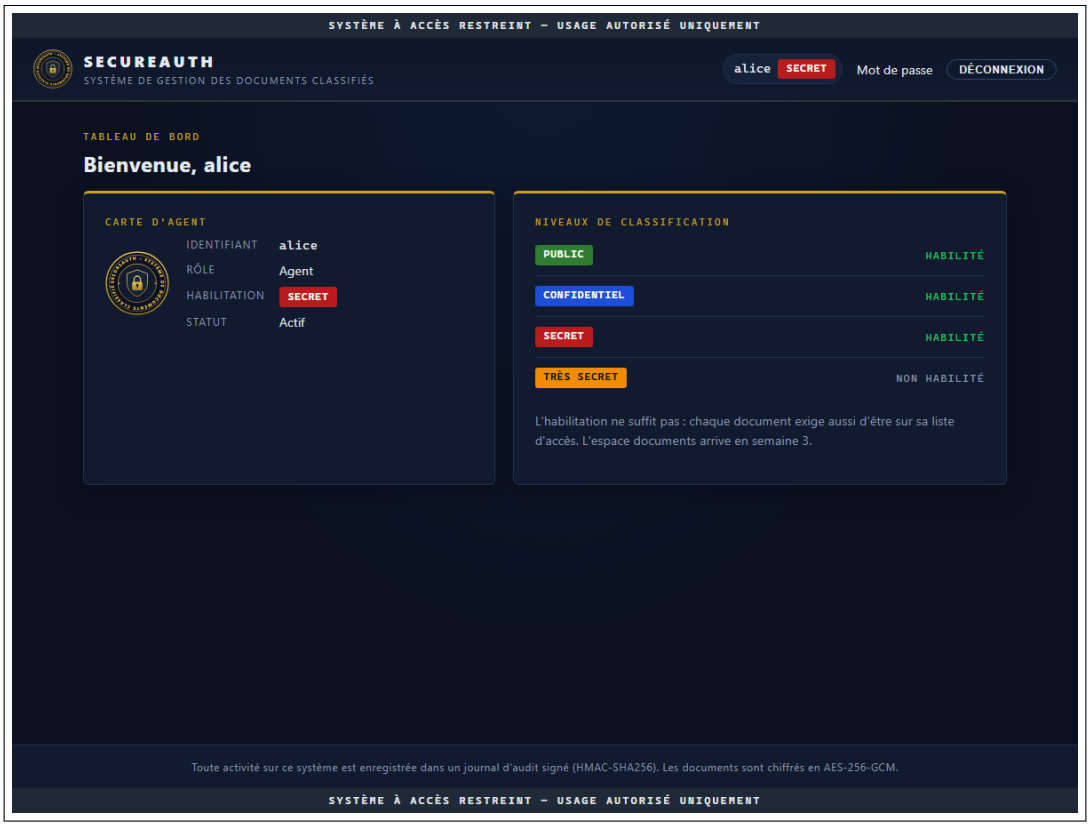


FIGURE 4.4 – Tableau de bord d'un agent connecté

Après la connexion, l'agent voit sa carte d'identité (identifiant, rôle, habilitation, statut) et les quatre niveaux de classification, avec pour chacun la mention « habilité » ou « non habilité ». La page rappelle que l'habilitation ne suffit pas : il faut aussi figurer sur la liste d'accès du document. L'espace documents sera ajouté en semaine 3.

4.4 Changement de Mot de Passe

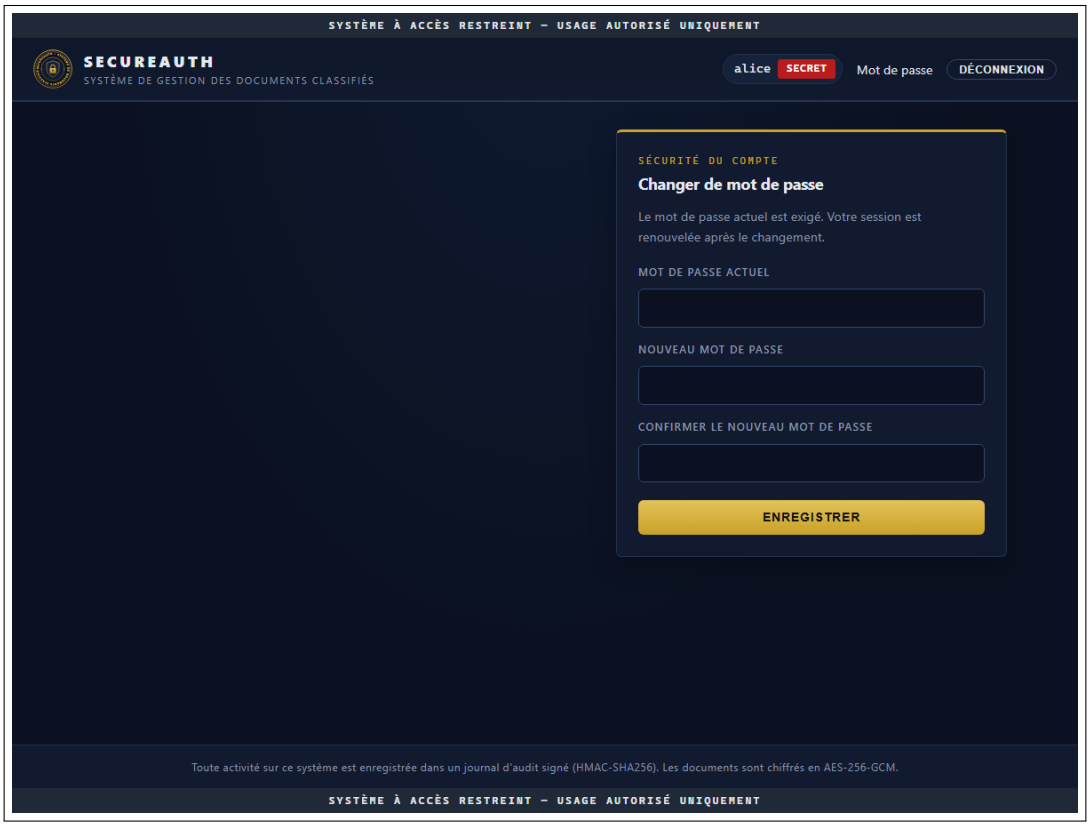


FIGURE 4.5 – Changement de mot de passe

Pour changer son mot de passe, l'utilisateur doit saisir l'ancien. Le nouveau doit respecter la même politique qu'à l'inscription et être différent de l'ancien. Après le changement, l'identifiant de session est renouvelé : un cookie volé avant le changement ne sert plus à rien.

4.5 Espace Documents

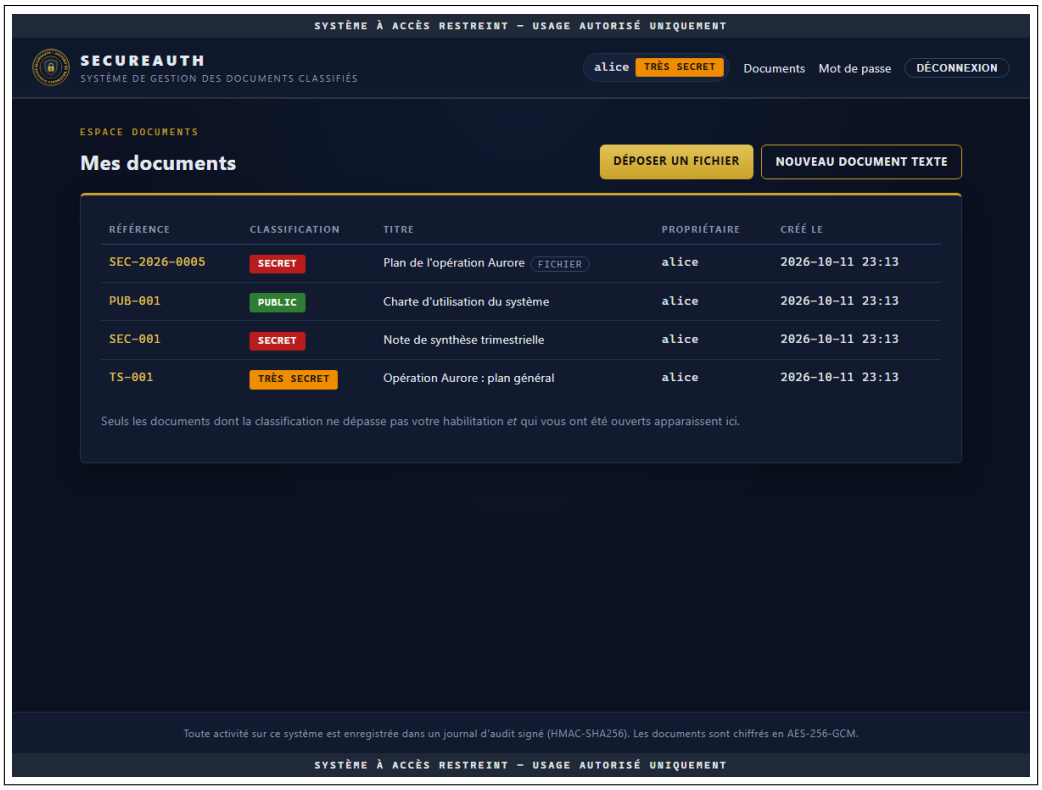


FIGURE 4.6 – Liste des documents lisibles par alice

La liste ne montre que les documents que l'agent peut lire : classification inférieure ou égale à son habilitation, et présence sur la liste d'accès. Le titre est déchiffré à l'affichage ; la référence, elle, est stockée en clair. Les adresses des documents utilisent un identifiant aléatoire (UUID) et jamais le numéro interne.

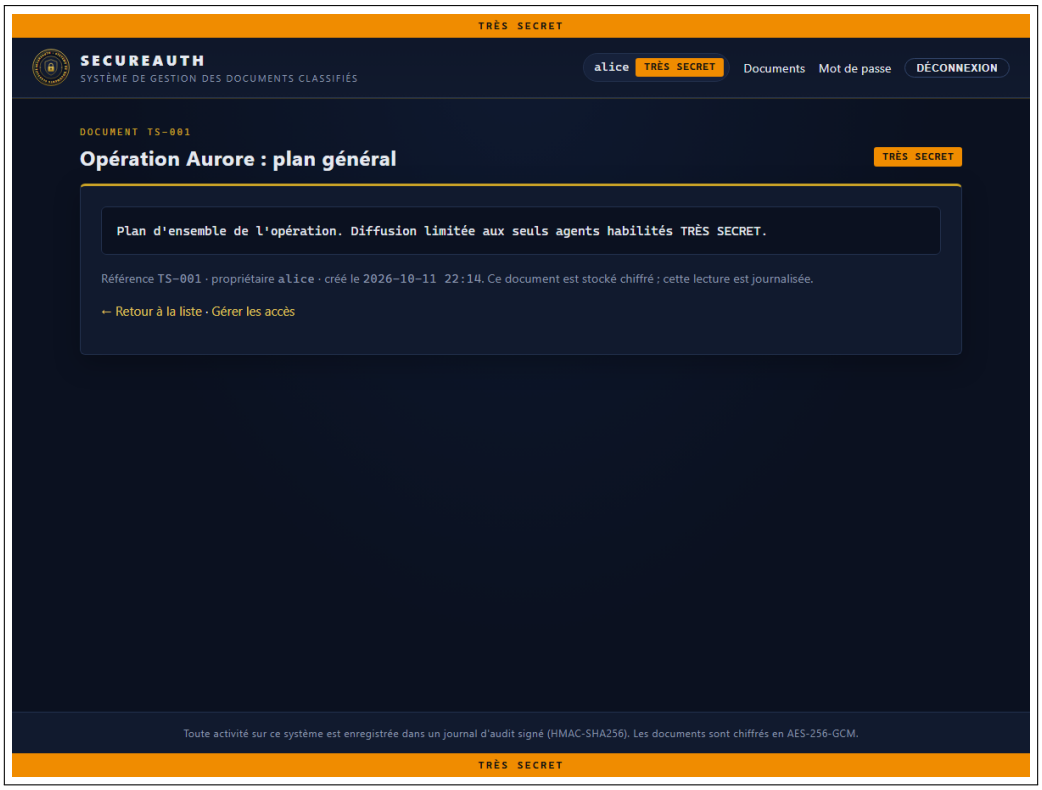


FIGURE 4.7 – Lecture d'un document TRÈS SECRET

À la lecture, le contenu est déchiffré puis affiché. Un bandeau de la couleur de la classification (orange pour TRÈS SECRET) encadre la page en haut et en bas. La lecture est inscrite dans le journal.

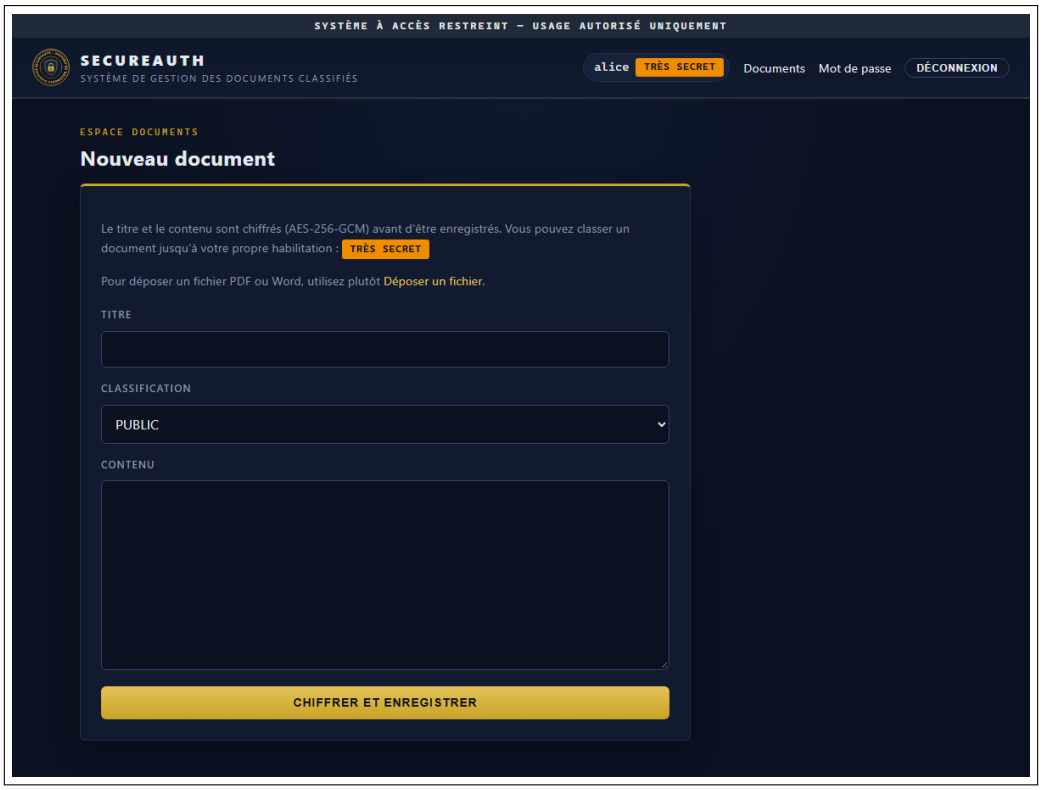


FIGURE 4.8 – Création d'un document

Le formulaire de création ne propose que les niveaux jusqu'à l'habilitation de l'agent. Le serveur revérifie : un niveau forcé dans la requête est refusé et journalisé.

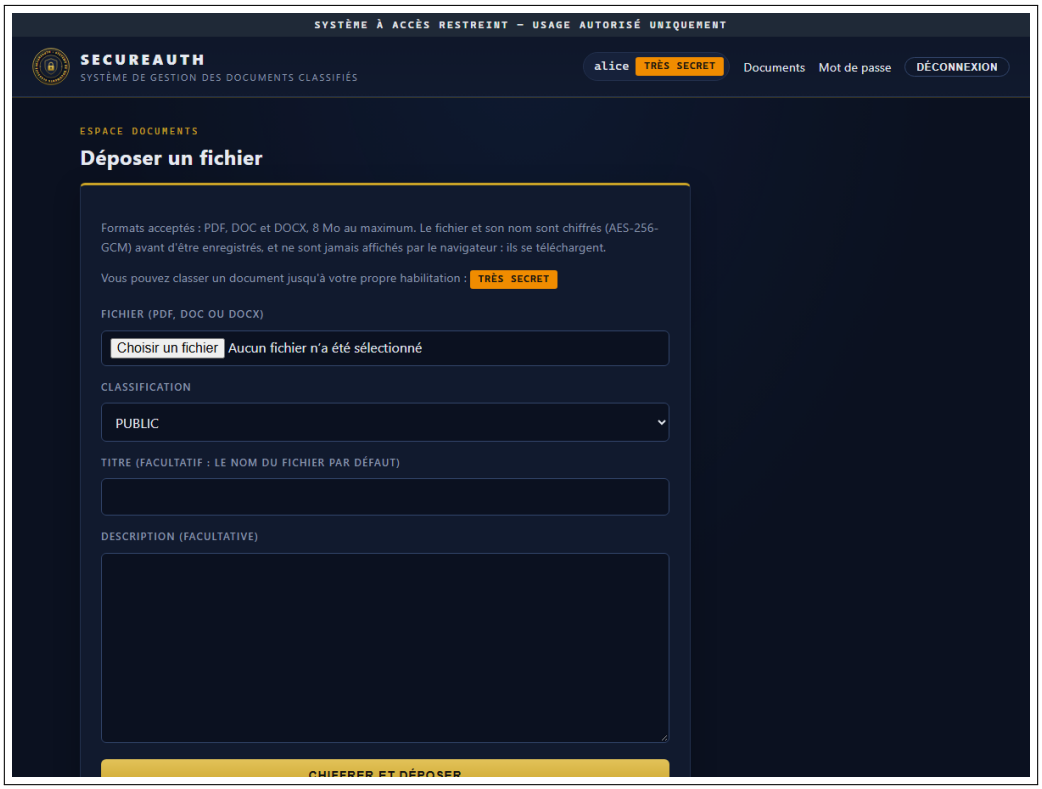


FIGURE 4.9 – Dépôt d'un fichier PDF, DOC ou DOCX

Le bouton « Déposer un fichier » de la page des documents ouvre ce formulaire. L'agent choisit le fichier, sa classification (jusqu'à son habilitation), un titre facultatif et une description facultative. Trois contrôles s'enchaînent côté serveur : l'extension doit être `pdf`, `doc` ou `docx`, la taille ne dépasse pas huit mégaoctets, et le début du contenu doit correspondre au format annoncé (en-tête `%PDF-` pour un PDF, signature OLE pour un DOC, archive contenant `word/` pour un DOCX). Un fichier `.pdf` qui n'est pas un PDF est donc refusé, et le refus est journalisé.

Le fichier est chiffré en AES-256-GCM avec son propre nonce, séparément du titre, de la description et du nom du fichier, qui sont chiffrés ensemble. Les données associées sont les mêmes que pour un document texte, avec un suffixe qui empêche de coller un fichier à la place des métadonnées, ou l'inverse.

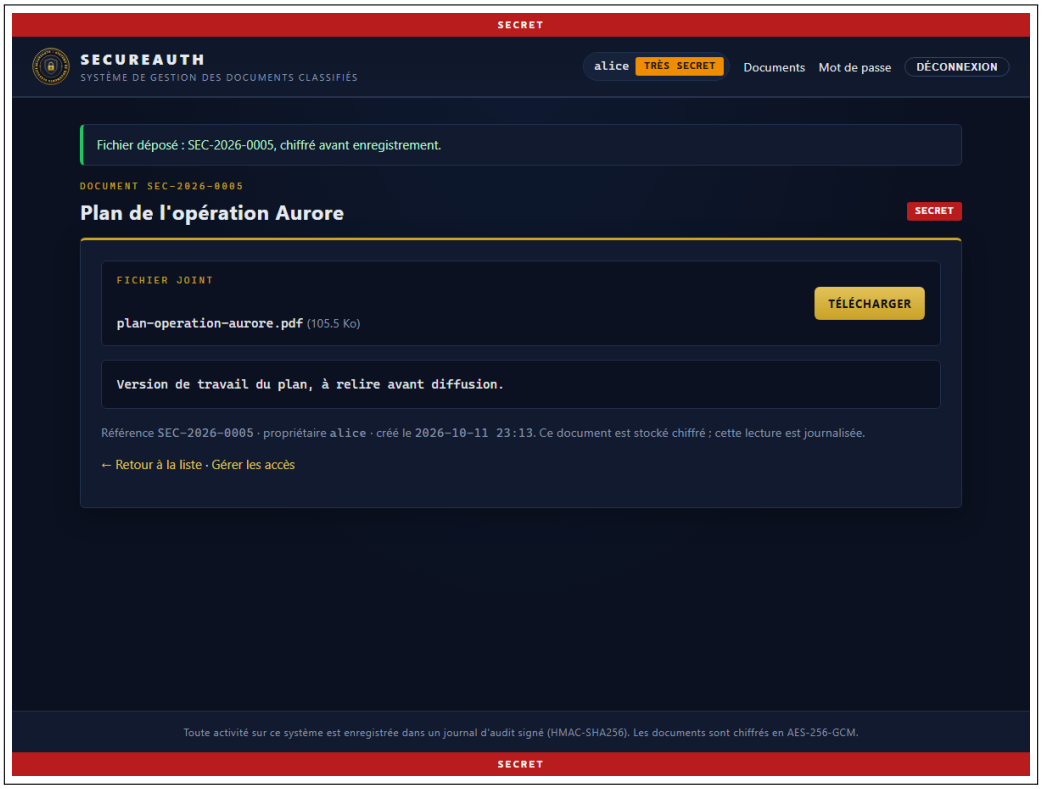


FIGURE 4.10 – Fiche d'un document avec fichier joint

La fiche affiche le nom et la taille du fichier avec un bouton de téléchargement. Le fichier n'est jamais ouvert dans le navigateur : il est servi en pièce jointe, avec un type fixé par le serveur d'après l'extension et l'en-tête `X-Content-Type-Options: nosniff`. Le contrôle ne garantit que le format, pas l'innocuité : un PDF bien formé peut contenir du code actif, et aucun antivirus n'analyse les dépôts. Chaque téléchargement est inscrit dans le journal (événement `DOC_DOWNLOAD`) sans le nom ni le contenu.

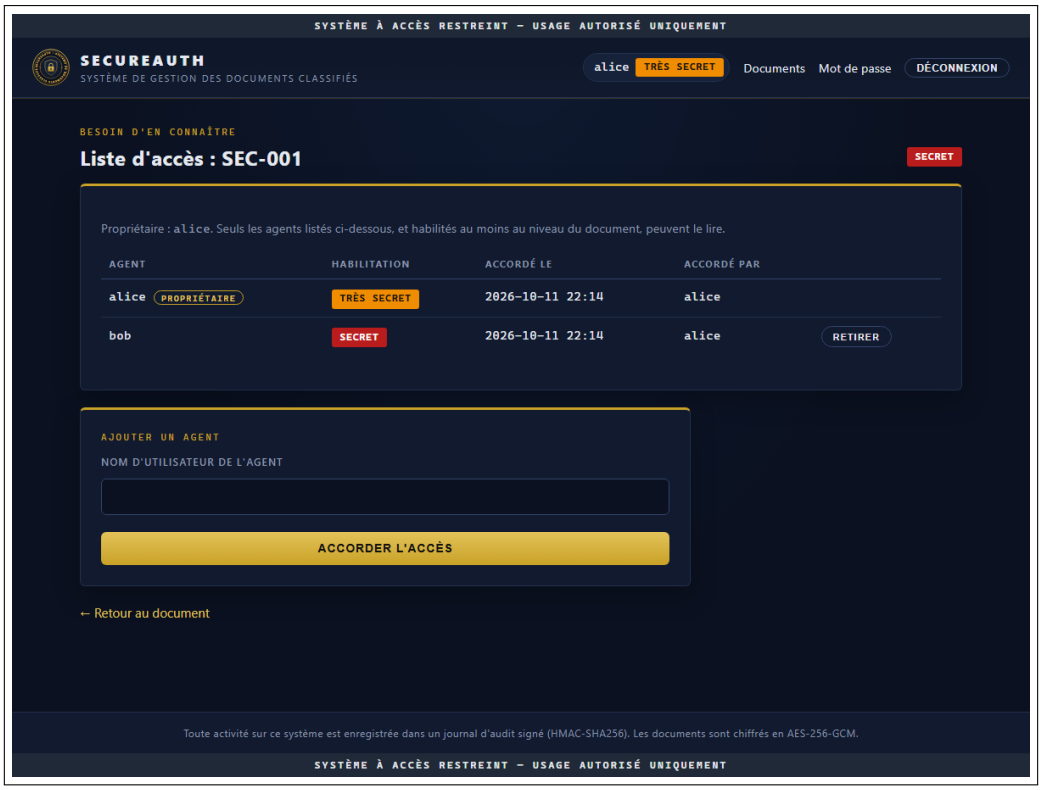


FIGURE 4.11 – Liste d'accès d'un document (partage)

Le propriétaire (ou l'administrateur) ajoute ou retire des agents. Un agent insuffisamment habilité est refusé, et le propriétaire ne peut jamais perdre son propre accès.

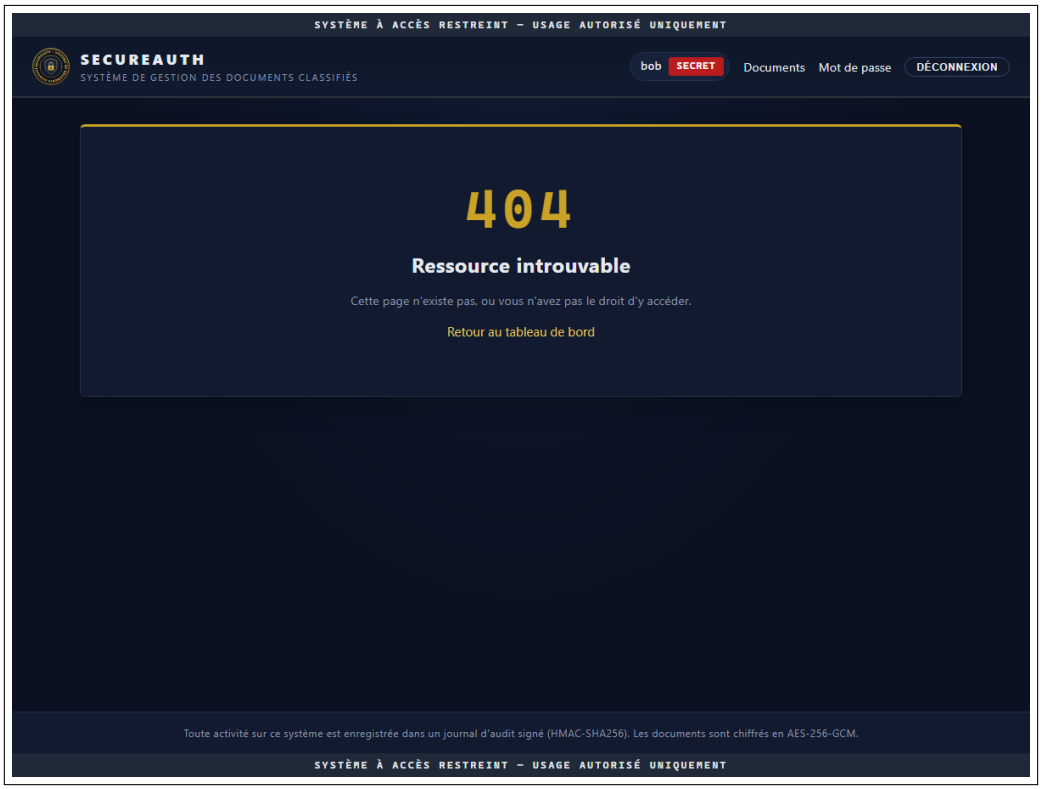


FIGURE 4.12 – Accès refusé : page 404 identique à un document inexistant

Quand bob (SECRET) force l'adresse de TS-001, il obtient la même page 404 que pour un document qui n'existe pas. Il ne peut donc pas savoir que ce document existe. Le refus, lui, est inscrit dans le journal.

4.6 Espace Administration

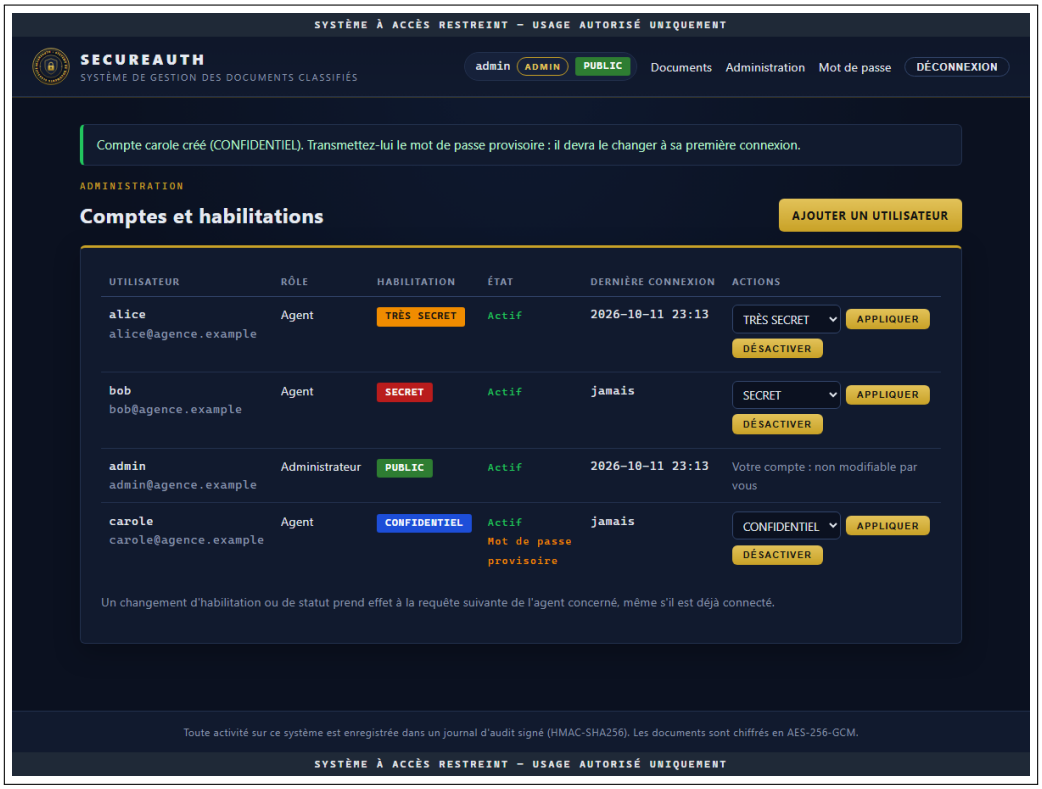


FIGURE 4.13 – Comptes et habilitations

L'administrateur voit les comptes, modifie une habilitation ou active et désactive un compte. Son propre compte n'est pas modifiable par lui. Un changement prend effet à la requête suivante de l'agent concerné, même s'il est déjà connecté.

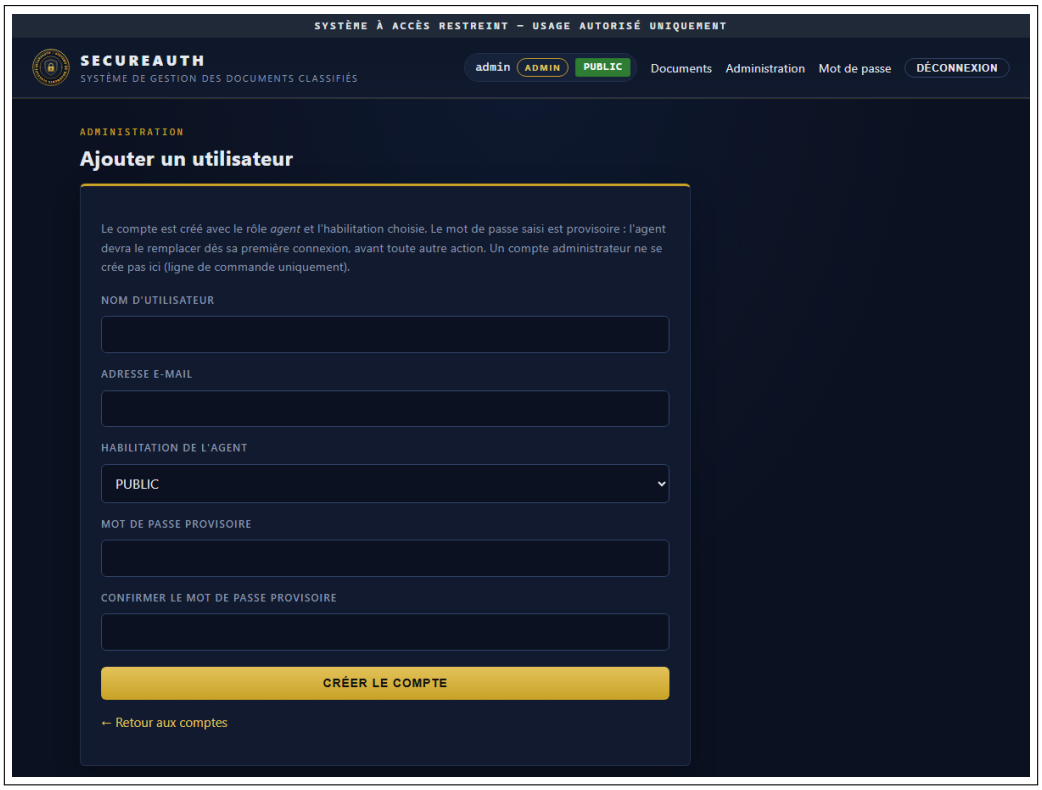


FIGURE 4.14 – Ajout d'un utilisateur par l'administrateur

Le bouton « Ajouter un utilisateur » ouvre ce formulaire. L'administrateur saisit le nom, l'adresse e-mail, l'**habilitation** de l'agent (de PUBLIC à TRÈS SECRET) et un mot de passe provisoire, soumis à la même politique que le mot de passe d'une inscription. Le compte créé est toujours un compte agent : un administrateur ne se crée que par la ligne de commande, et un champ « rôle » ajouté à la requête est ignoré. La création est inscrite dans le journal (`USER_CREATE`) sans le mot de passe.

Le mot de passe provisoire est connu de l'administrateur, ce qui est contraire à l'idée qu'un mot de passe n'appartient qu'à son titulaire. Le compte porte donc un indicateur « mot de passe provisoire » (visible dans la liste) : à sa première connexion, l'agent est redirigé vers le changement de mot de passe et ne peut rien faire d'autre, sauf se déconnecter, tant qu'il ne l'a pas remplacé.

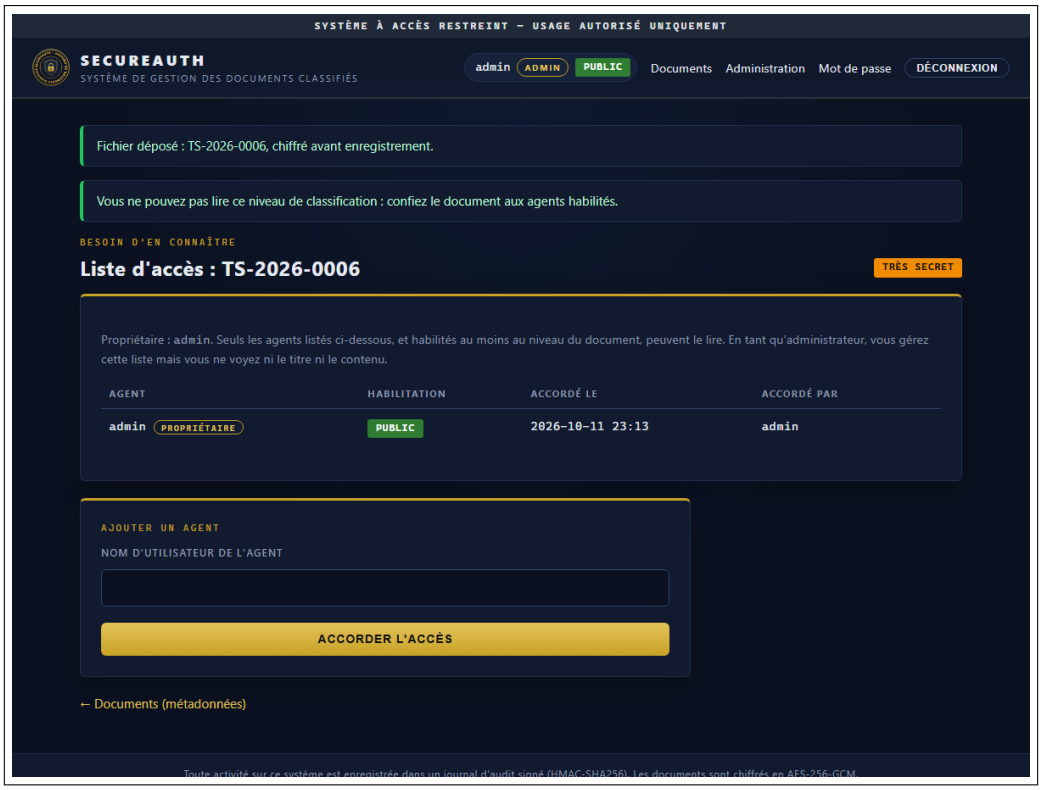


FIGURE 4.15 – Dépôt par l'administrateur : classification TRÈS SECRET, puis partage

L'administrateur peut aussi déposer un document et il choisit sa classification à tous les niveaux, alors qu'un agent est limité à son habilitation. Son habilitation reste pourtant PUBLIC : il ne peut pas lire un document classé au-dessus (pas de lecture vers le haut, et le dépôt n'ajoute aucun droit). Après un dépôt à un niveau qu'il ne peut pas lire, il est donc conduit directement à la liste d'accès pour confier le document à des agents habilités. Un agent insuffisamment habilité est refusé, et l'administrateur ne peut pas s'ajouter lui-même.

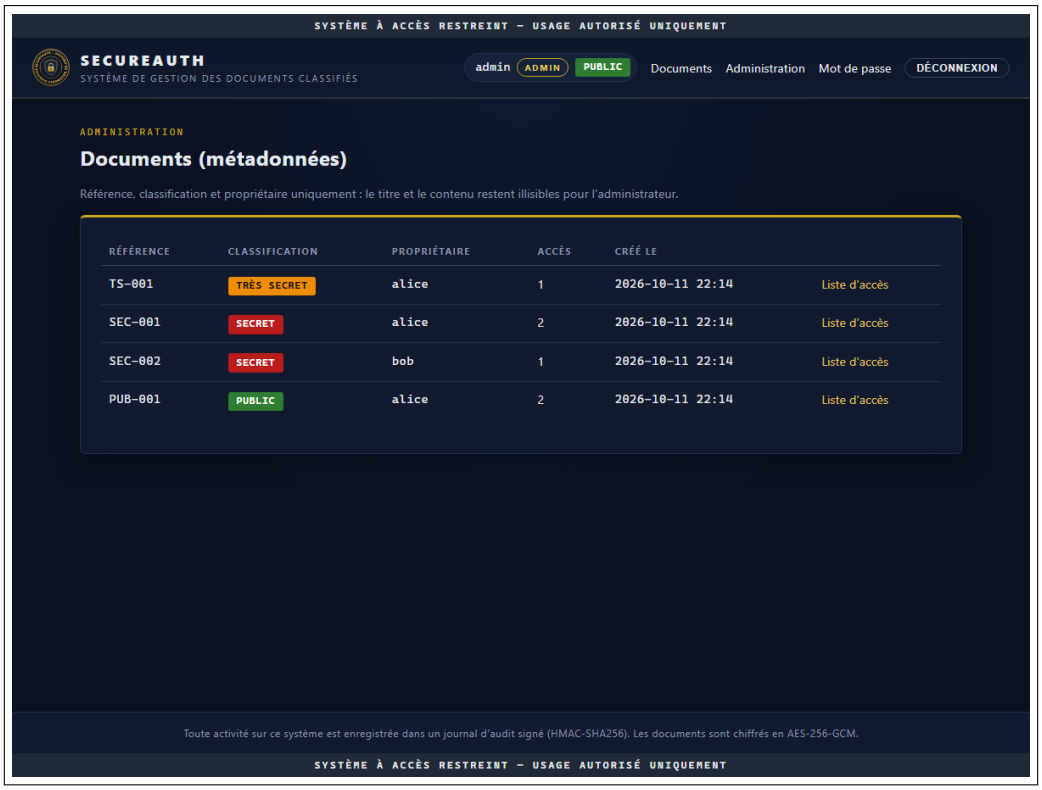


FIGURE 4.16 – Documents : métadonnées uniquement

La page des documents donne la référence, la classification, le propriétaire et le nombre d'accès, mais jamais le titre ni le contenu.

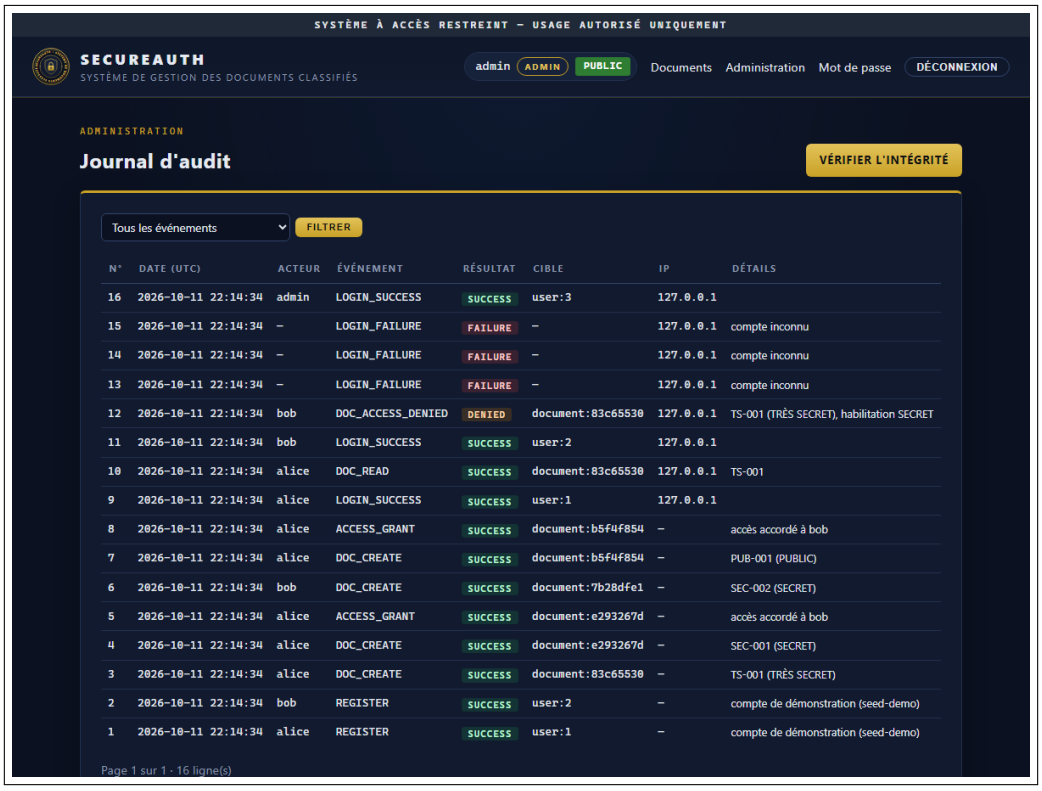


FIGURE 4.17 – Journal d'audit

Le journal est paginé et filtrable par type d'événement. Chaque ligne indique l'acteur, l'événement, le résultat, la cible, l'adresse IP et un détail qui ne contient jamais de mot de passe, de contenu ni de clé.

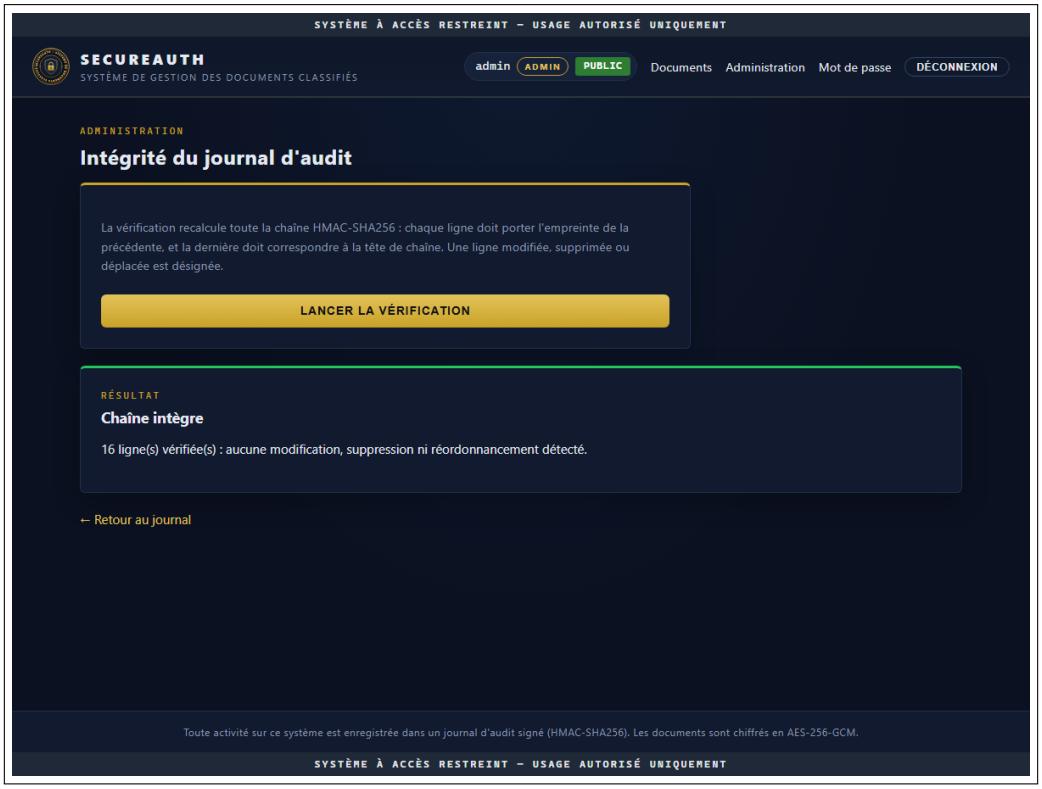


FIGURE 4.18 – Vérification du journal : chaîne intègre

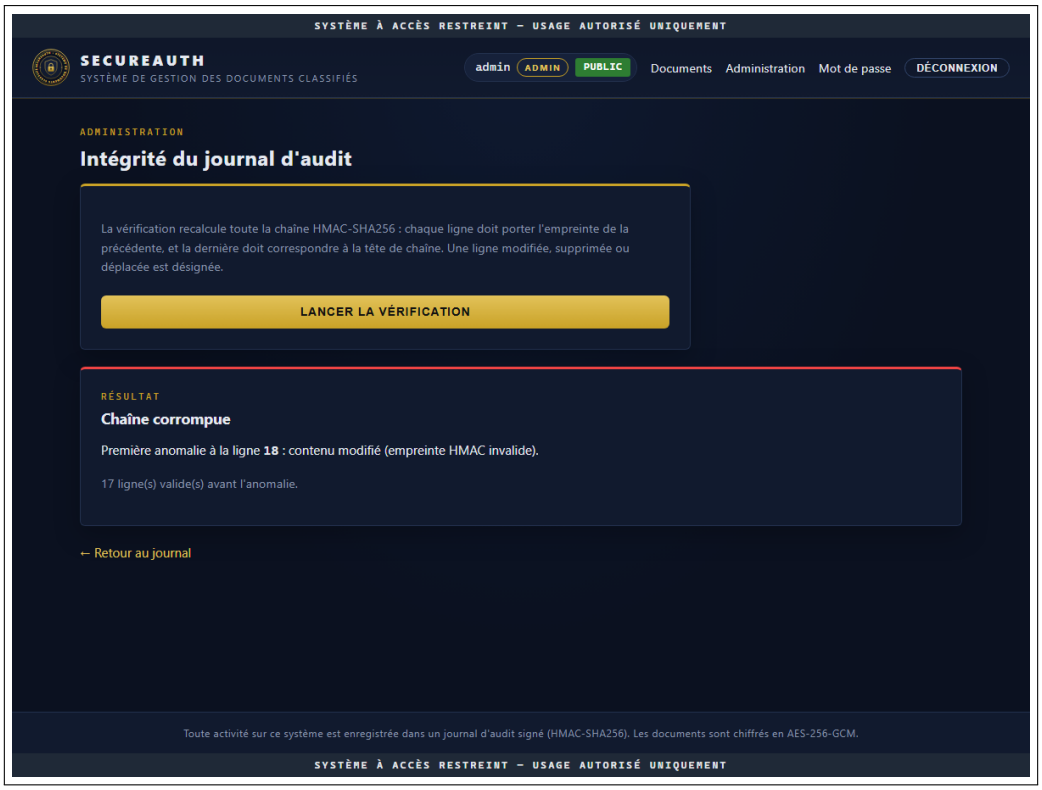


FIGURE 4.19 – Vérification du journal après modification d'une ligne en base

La vérification recalcule toute la chaîne HMAC. Quand une ligne a été modifiée directement en base, la page désigne la première ligne en cause (ici la ligne 18) et indique combien de lignes restaient valides avant elle.

Chapitre 5

Tests de Sécurité

5.1 Méthode

Le cahier des charges fixe vingt-cinq cas de test de sécurité, de T01 à T25. L'équipe en a ajouté cinq (T26 à T30) pour les fonctions apparues ensuite : comptes créés par l'administrateur, dépôt de fichiers, classification choisie par l'administrateur. Chacun est vérifié de deux façons.

— **Des tests automatiques** (`pytest`), qui tournent sans conteneur avec une base en mémoire et un faux Redis. Ils couvrent les mots de passe, l'authentification, le verrouillage, les sessions, les règles d'accès, le chiffrement, les documents, le partage, l'administration, le journal d'audit, la limitation de débit, le jeu de démonstration et la configuration de l'infrastructure.

— **Un script contre l'application réellement déployée** (`live_checks.py`) : HTTPS, Nginx, MariaDB et Redis en marche. Il rejoue ce qu'on ferait à la main avec `curl`, `openssl` et Burp Repeater : analyse du cookie, tentative de connexion en TLS 1.1, accès à un document d'un autre agent, modification d'une ligne du journal directement dans la base, rafale de connexions.

Les résultats complets sont dans `docs/tests-securite.md` et `docs/preuves/`. Les captures d'écran de Burp Suite restent à réaliser : le script envoie les mêmes requêtes, mais il ne les remplace pas dans le rapport final.

5.2 Résultats

TABLE 5.1 – Les trente cas de test de sécurité (T26 à T30 : ajouts de l'équipe)

Cas	Scénario	Résultat attendu	Résultat
T01	Compte inexistant vs mauvais mot de passe	Même message, durée comparable	Conforme
T02	6 échecs consécutifs puis le bon mot de passe	Compte verrouillé 15 min, même message	Conforme
T03	Injection SQL dans le formulaire (et les autres entrées)	Échec normal, aucune erreur SQL	Conforme

Cas	Scénario	Résultat attendu	Résultat
T04	Drapeaux du cookie de session	Secure, HttpOnly, SameSite=Lax, préfixe ___ Host-	Conforme
T05	Identifiant avant / après connexion	Identifiant différent	Conforme
T06	Rejouer un cookie après déconnexion	Redirection vers la connexion	Conforme
T07	Inactivité supérieure à 30 minutes	Session expirée	Partiel
T08	POST sans jeton	HTTP 400	Conforme
T09	Agent SECRET vers un document TRÈS SECRET	404 + DOC_ACCESS_ DENIED	Conforme
T10	Habilitation suffisante mais hors liste d'accès	404	Conforme
T11	IDOR : identifiant d'un autre document	404 identique à un document inexistant	Conforme
T12	L'administrateur tente de lire un document	404	Conforme
T13	L'administrateur s'attribue un accès ou modifie sa propre habilitation	Refus + ADMIN_SELF_ MODIFY_DENIED	Conforme
T14	Un agent appelle une route /admin/*	403	Conforme
T15	Habilitation abaissée après attribution	Accès perdu immédiatement	Conforme
T16	Création au-dessus de son habilitation (champ modifié)	Refus	Conforme
T17	Classification modifiée directement en base	Déchiffrement impossible + INTEGRITY_FAILURE	Conforme
T18	Lecture des documents directement en base	Uniquement du binaire illisible	Conforme
T19	Ligne du journal modifiée en base	Ligne signalée	Conforme
T20	Ligne du journal supprimée (milieu ou fin)	Rupture détectée	Conforme
T21	Balise de script dans un titre de document	Affichée comme texte, bloquée par la CSP	Conforme
T22	Connexion en TLS 1.1	Refusée	Conforme
T23	En-têtes de sécurité	Tous présents	Conforme
T24	Accès direct aux ports 3306 et 6379	Injoignables depuis l'hôte	Conforme
T25	Rafale de connexions	HTTP 429 après le seuil	Conforme

Cas	Scénario	Résultat attendu	Résultat
T26	L'administrateur ajoute un agent ; l'agent ouvre le formulaire ; première connexion	Agent refusé (403) ; compte créé avec mot de passe provisoire ; changement imposé ; USER_CREATE sans mot de passe	Conforme
T27	Dépôt d'un PDF de 3 Mo puis lecture directe de la colonne en base	Binaire illisible, ni contenu ni nom en clair ; téléchargement identique en pièce jointe	Conforme
T28	Téléchargement hors liste d'accès, après partage, puis classification modifiée en base	404 + DOC_ACCESS_ DENIED ; 200 après partage ; 500 + INTEGRITY_ FAILURE après falsification	Conforme
T29	Faux PDF, exécutable, page HTML, fichier de 11 Mo, corps de 3 Mo sur une autre page	400 pour les fichiers invalides, 413 pour les envois trop gros, rien d'enregistré	Conforme
T30	L'administrateur dépose un document TRÈS SECRET puis tente de le lire	Dépôt accepté, redirection vers le partage, lecture et téléchargement refusés (404), partage seulement aux agents habilités	Conforme

Vingt-neuf cas sur trente sont conformes. Le cas T07 (inactivité de plus de trente minutes) est partiel : la durée absolue de huit heures est testée, et le cookie ainsi que la clé Redis expirent bien après trente minutes, mais l'attente réelle de trente minutes n'a pas été rejouée. Le cas T20 (suppression d'une ligne du journal) est démontré par les tests automatiques seulement, pour ne pas détruire la chaîne du journal de la base réelle : une ligne signée supprimée ne peut pas être reconstituée.

5.3 Défauts découverts et corrigés

Les tests contre les vrais services ont fait apparaître des défauts que les tests automatiques ne pouvaient pas voir.

— Le certificat du serveur n'avait pas d'identifiant de clé de l'autorité (*Authority Key Identifier*) : les clients stricts, comme Python 3.14, le refusaient. Le script de génération de la mini-PKI ajoute maintenant ces identifiants, et le certificat a été réémis avec la même clé.

— La première limite de débit de Nginx comptait aussi les pages affichées : ouvrir le formulaire de connexion consommait le quota. Seules les requêtes `POST` sont désormais comptées.

— Le verrouillage du compte, prévu en semaine 2, n'était pas encore codé : le champ existait mais aucune route ne l'utilisait. Il est implémenté et testé.

— La création d'un document échouait sur MariaDB (*Data too long for column*) alors qu'elle passait avec SQLite : la référence provisoire était plus longue que la colonne. La base en mémoire des tests automatiques n'applique pas les longueurs, seul le

test contre la vraie base l'a révélé ; la référence est raccourcie et deux tests de non-régression ont été ajoutés.

Une observation sur l'environnement : sur la machine de développement, `localhost:3306` répond, car c'est le MariaDB de XAMPP et non le conteneur. La preuve que Docker ne publie rien est donc la colonne des ports de `docker compose ps`, pas un test de connexion depuis l'hôte.

5.4 Durcissement

En semaine 5, plusieurs protections ont été ajoutées :

— limitation de débit par adresse IP, à deux niveaux (Flask-Limiter avec compteurs dans Redis, et `limit_req` dans Nginx) ; au-delà du seuil, la réponse est HTTP 429 ;
— HSTS avec une durée courte de cinq minutes, pour ne pas piéger `localhost` ;
— conteneur applicatif en lecture seule, sans capacités Linux et sans élévation de privilèges ;
— en-tête `Cache-Control: no-store` sur toute page affichée à un agent connecté, pour que le bouton « Précédent » ne rouvre pas un document après la déconnexion ;
— des tests d'infrastructure qui échouent si la configuration livrée s'affaiblit ;
— pour le dépôt de fichiers : liste blanche d'extensions, contrôle de la signature du contenu, taille limitée à huit mégaoctets, nom de fichier nettoyé, téléchargement en pièce jointe uniquement. La limite de corps de requête de Nginx reste à deux mégaoctets partout, sauf sur la seule adresse de dépôt (dix mégaoctets), et l'application relève sa propre limite pour cette seule page.

Chapitre 6

Conclusion et Perspectives

6.1 Bilan du Projet

Les six semaines du projet sont réalisées. L'infrastructure fonctionne en quatre conteneurs avec un réseau interne et du TLS signé par une mini-autorité locale. L'authentification est complète : inscription, connexion avec un message d'erreur unique, verrouillage du compte après cinq échecs, sessions côté serveur renouvelées à la connexion, changement de mot de passe. L'autorisation repose sur un module unique qui applique la lecture sans montée de niveau, le besoin d'en connaître et la séparation des rôles. Les documents sont chiffrés en AES-256-GCM avec des données associées qui scellent leur classification. Le journal d'audit est chaîné par HMAC-SHA256 et vérifiable depuis la ligne de commande comme depuis l'interface. L'administrateur peut ouvrir des comptes agents avec l'habilitation de son choix (mot de passe provisoire à changer immédiatement), et les agents peuvent déposer des fichiers PDF, DOC et DOCX, chiffrés à part et téléchargeables seulement par ceux qui ont le droit de les lire. Plus de trois cents tests automatiques passent, et les attaques ont été rejouées contre l'application déployée.

Points forts :

— les mots de passe sont protégés par Argon2id et jamais conservés en clair ;
— un document interdit répond par la même page 404 qu'un document inexistant ;
— l'administrateur gère les comptes mais ne peut lire aucun document, ni modifier ses propres droits ;
— changer la classification d'un document directement en base le rend illisible au lieu de le livrer ;
— le rôle et l'habilitation sont relus en base à chaque requête : une désactivation ou une baisse d'habilitation prend effet immédiatement ;
— toute modification, suppression ou permutation d'une ligne du journal est détectée, et la page d'administration désigne la première ligne en cause ;
— l'application refuse de démarrer si une clé est absente, mal formée ou si les deux clés sont identiques ;
— un fichier déposé n'est jamais en clair en base ni dans le journal, son format est vérifié sur son contenu et pas seulement sur son nom, et l'administrateur qui choisit la classification d'un dépôt ne gagne aucun droit de lecture.

Ce qui reste à faire : les captures d'écran de Burp Suite pour le rapport final, l'attente réelle de trente minutes pour le cas T07, et les points listés dans les perspectives.

6.2 Leçons Apprises

— **Les détails de sécurité se cachent dans le temps de réponse.** Pour ne pas révéler l'existence d'un compte, il faut calculer l'empreinte Argon2id même quand le compte n'existe pas, et le faire avant le test d'unicité à l'inscription. La mesure sur le service réel donne environ 125 ms pour un compte inconnu et 115 ms pour un mauvais mot de passe.
— **Une bibliothèque peut ne rien faire sans prévenir.** Avec Flask-Session, la régénération d'une session vide ne fait rien : il faut écrire l'identifiant de l'utilisateur dans la session avant de la régénérer, sinon la protection contre la fixation de session est inopérante.
— **Une signature est exigeante.** Le journal d'audit est déclaré corrompu si la date relue diffère d'une microseconde de la date signée : la colonne doit donc stocker les microsecondes.
— **Tester en vrai ne donne pas les mêmes résultats que tester en mémoire.** Le certificat sans identifiant d'autorité, la limite Nginx qui comptait les GET et le port 3306 de XAMPP n'apparaissent qu'avec les vrais services.
— **Un contexte partagé peut masquer des erreurs.** Flask-WTF garde le jeton CSRF dans un objet lié au contexte d'application : si plusieurs clients de test partagent ce contexte, le deuxième échoue pour une raison qui n'existe pas en production. Les tests donnent donc un contexte à chaque requête.
— **L'environnement de développement compte.** Sous Windows, Apache de XAMPP occupait déjà les ports 80 et 443 ; `curl` est un alias dans PowerShell (il faut écrire `curl.exe`) ; et la vérification de révocation du certificat échoue car l'autorité locale n'a pas de liste de révocation.

6.3 Perspectives d'Évolution

— authentification multifacteur ;
— analyse antivirus des fichiers déposés (par exemple avec ClamAV dans un conteneur dédié) et conversion en un format sans code actif ;
— réinitialisation du mot de passe par lien à usage unique, pour que l'administrateur n'ait plus à connaître de mot de passe provisoire ;
— rotation des clés et stockage dans un coffre-fort de secrets ou un module matériel (HSM) ;
— règle *no write down* du modèle Bell-LaPadula et reclassification contrôlée des documents ;
— protection des habilitations par un tag HMAC, pour détecter une modification directe en base ;
— liste de révocation (CRL ou OCSP) pour l'autorité de certification ;
— déverrouillage d'un compte par l'administrateur et notification de l'utilisateur ;
— HSTS d'un an et certificat émis par une autorité reconnue, pour une mise en production ;
— migrations de schéma (Alembic) et API publique, aujourd'hui hors périmètre.

6.4 Conclusion Générale

SecureAuth part d'une idée simple : une application sécurisée ne repose jamais sur une seule protection. Ici, l'authentification identifie l'utilisateur, l'autorisation décide de ce qu'il peut lire, la cryptographie protège les documents même si la base est lue directement, et le journal signé permet de prouver ce qui s'est passé.

Le projet montre aussi qu'une protection n'existe vraiment que si on l'a attaquée. Les tests automatiques ont fixé le comportement attendu, puis les mêmes attaques rejouées contre les services en marche ont révélé des défauts de configuration que le code seul ne montrait pas. Les limites restantes sont connues et écrites : un attaquant qui modifie directement l'habilitation d'un compte en base n'est pas détecté, la détention de la clé HMAC permettrait de refaire une chaîne valide, et l'autorité de certification locale ne peut pas révoquer un certificat. Chaque décision de sécurité du projet peut ainsi être justifiée et démontrée par une preuve, ce qui était l'objectif du module.

Chapitre A

Glossaire

AAD Données associées : informations non chiffrées mais protégées par AES-GCM ; si on les modifie, le déchiffrement échoue.

AES-256-GCM Chiffrement symétrique authentifié : il garantit à la fois la confidentialité et l'intégrité.

Argon2id Fonction de hachage de mots de passe, volontairement lente et gourmande en mémoire pour résister à la force brute.

Bell-LaPadula Modèle de confidentialité : pas de lecture vers le haut, pas d'écriture vers le bas.

Besoin d'en connaître Principe selon lequel l'habilitation ne suffit pas : il faut une autorisation explicite pour chaque document.

Classification Niveau de sensibilité d'un document : PUBLIC, CONFIDENTIEL, SECRET ou TRÈS SECRET.

CSRF Attaque qui fait exécuter une action à un utilisateur connecté depuis un autre site.

Fixation de session Attaque qui impose à la victime un identifiant de session connu de l'attaquant.

Habilitation Niveau maximal de classification qu'un utilisateur est autorisé à lire.

HMAC Code d'authentification de message à clé secrète, qui prouve l'intégrité d'une donnée.

IDOR Accès à l'objet d'un autre utilisateur en changeant son identifiant dans la requête.

Nonce Valeur aléatoire utilisée une seule fois par chiffrement.

PKI Infrastructure de clés publiques : ici, une mini-autorité de certification locale créée avec OpenSSL.

TLS Protocole qui chiffre la communication entre le navigateur et le serveur.

Contact :
Pour toute question concernant ce projet, veuillez contacter l'équipe de développement SecureAuth.
TEK-UP – Année universitaire 2026–2027